

OCR WORD SEARCH SOLVER

Rapport Technique Final

GROUPE DE PROJET S3

Mezroui Ayoub	<i>Détection & Segmentation</i>
Issac Kaba	<i>Prétraitement & Vision</i>
Chahine Nabi	<i>Réseau de Neurones & Solver</i>
Yanis Jerjini	<i>Interface Utilisateur (GUI)</i>

École EPITA

14 décembre 2025

Remerciements

Nous tenons tout d'abord à remercier l'équipe pédagogique de l'EPITA et plus particulièrement notre responsable de projet, M. David Bouchet, pour l'encadrement et les ressources mises à notre disposition tout au long de ce semestre.

Ce projet a été l'occasion pour nous de mettre en pratique les compétences acquises en programmation C, en mathématiques appliquées et en gestion de projet. Il représente l'aboutissement de trois mois de travail intensif et de collaboration.

Table des matières

1	Introduction Générale	5
1.1	Contexte du Projet	5
1.2	Cahier des Charges et Contraintes	6
1.3	Répartition des Rôles	7
1.3.1	Pôle 1 : Prétraitement (Issac Kaba)	7
1.3.2	Pôle 2 : Détection (Mezroui Ayoub)	7
1.3.3	Pôle 3 : Intelligence Artificielle (Chahine Nabi)	7
1.3.4	Pôle 4 : Interface (Yanis Jerjini)	7
2	Prétraitement de l'Image	8
2.1	Vue d'ensemble du pipeline	9
2.2	Conversion en Niveaux de Gris	10
2.2.1	Formule Mathématique	10
2.2.2	Implémentation C	10
2.3	Réduction du Bruit (Denoising)	11
2.4	Correction Automatique de la Rotation (Deskew)	12
2.4.1	Approche par Projection de Variance	12
2.4.2	Rotation Matricielle	12
2.5	Binarisation Adaptative	13
2.5.1	Méthode de Niblack	13
2.5.2	Optimisation : Image Intégrale	13
3	Détection et Segmentation	14
3.1	Analyse des Composantes Connexes	15
3.1.1	Structure de Données	15
3.2	Filtrage et Nettoyage	16
3.3	Séparation Grille / Liste de Mots	17
3.3.1	Algorithme de Clustering	17
3.4	Découpage des Caractères (Slicing)	18

3.5	Reconstruction de la Grille	19
4	Intelligence Artificielle et OCR	20
4.1	Introduction	21
4.2	Architecture du Modèle	22
4.2.1	Topologie	22
4.3	Mathématiques du Réseau	23
4.3.1	Propagation Avant (Forward Propagation)	23
4.3.2	Rétropropagation (Backpropagation)	23
4.4	Dataset et Entraînement	24
4.5	Preuve de Concept : Le XOR	25
5	Résolution de la Grille (Solver)	26
5.1	Algorithme de Recherche	27
5.1.1	Les 8 Directions	27
5.2	Gestion des Erreurs OCR	28
6	Interface Utilisateur et Intégration	29
6.1	Conception de l'Interface	30
6.1.1	Architecture Logicielle	30
6.2	Fonctionnalités Implémentées	31
6.2.1	1. Chargement et Affichage	31
6.2.2	2. Visualisation des Étapes Intermédiaires	31
6.2.3	3. Édition et Correction	31
6.3	Dessin Vectoriel avec Cairo	32
7	Conclusion	33
7.1	Bilan Technique	34
7.2	Améliorations Possibles	35
A	Guide d'Utilisation	36
A.1	Compilation	36
A.2	Exécution	37
B	Bibliographie	38

Table des figures

Chapitre 1

Introduction Générale

1.1 Contexte du Projet

L'Optical Character Recognition (OCR), ou reconnaissance optique de caractères, est une technologie permettant de convertir différents types de documents tels que des documents papiers scannés, des fichiers PDF ou des photos numériques en fichiers modifiables et consultables.

Dans le cadre de notre scolarité à l'EPITA, il nous a été demandé de réaliser, par groupe de quatre, une application complète capable de résoudre une grille de mots mêlés à partir d'une simple image. Ce projet, nommé "OCR Word Search Solver", combine plusieurs domaines de l'informatique :

- Le traitement d'image (Computer Vision).
- L'intelligence artificielle (Réseaux de Neurones).
- L'algorithmique avancée.
- Le développement d'interfaces graphiques.

L'objectif final est de fournir un logiciel robuste, capable de prendre en entrée une photo prise par un smartphone (souvent imparfaite, bruitée ou inclinée) et de ressortir la grille résolue visuellement.

1.2 Cahier des Charges et Contraintes

Le projet doit respecter des contraintes techniques strictes définies dans le sujet :

- Langage de programmation : **C** exclusivement.
- Compilation : Environnement Linux avec **gcc** et les drapeaux **-Wall -Wextra**.
- Pas de bibliothèques "boîte noire" pour l'IA (interdiction d'utiliser TensorFlow ou OpenCV pour le réseau de neurones).
- Interface graphique obligatoire.

1.3 Répartition des Rôles

Pour mener à bien ce projet complexe, nous avons divisé les tâches selon quatre pôles d'expertise :

1.3.1 Pôle 1 : Prétraitement (Issac Kaba)

Issac est responsable de la "propreté" de l'image. Son rôle est de transformer l'image brute en une image binaire parfaite, corrigée de toute rotation et nettoyée de tout bruit. C'est la base indispensable du projet.

1.3.2 Pôle 2 : Détection (Mezroui Ayoub)

Ayoub s'occupe de la structure. Il doit "comprendre" l'image : où est la grille ? Où est la liste de mots ? Il découpe ensuite l'image en petites vignettes contenant chacune une lettre.

1.3.3 Pôle 3 : Intelligence Artificielle (Chahine Nabi)

Chahine est en charge du "cerveau". Il a développé un réseau de neurones multicouches (MLP) pour reconnaître les lettres, ainsi que l'algorithme de résolution de la grille (Solver).

1.3.4 Pôle 4 : Interface (Yanis Jerjini)

Yanis est l'architecte de l'interaction. Il assemble tous les modules dans une interface graphique ergonomique permettant à l'utilisateur final d'utiliser le logiciel simplement.

Chapitre 2

Prétraitement de l'Image

Par Issac Kaba

2.1 Vue d'ensemble du pipeline

Le prétraitement est une succession de filtres appliqués à l'image source. L'objectif est de normaliser les données pour faciliter le travail de segmentation et de reconnaissance. Les étapes sont les suivantes :

1. Chargement de l'image.
2. Conversion en niveaux de gris (Grayscale).
3. Réduction du bruit (Denoising).
4. Correction de l'inclinaison (Deskew).
5. Binarisation adaptative.

2.2 Conversion en Niveaux de Gris

L'image chargée est souvent en couleur (RGB). Pour l'OCR, la couleur est une information superflue qui augmente la complexité des calculs (3 canaux à traiter au lieu d'un). Nous utilisons la méthode de la luminance perceptuelle pour convertir l'image. Contrairement à la moyenne simple $((R + G + B)/3)$, cette formule tient compte de la sensibilité de l'œil humain.

2.2.1 Formule Mathématique

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (2.1)$$

On remarque que le coefficient du vert est le plus élevé, car l'œil humain est très sensible à cette couleur.

2.2.2 Implémentation C

L'implémentation se fait via la librairie SDL2 en manipulant directement les surfaces de pixels.

```
1 void grayscale(SDL_Surface* surface) {  
2     Uint32* pixels = (Uint32*)surface->pixels;  
3     for (int i = 0; i < surface->w * surface->h; i++) {  
4         Uint8 r, g, b;  
5         SDL_GetRGB(pixels[i], surface->format, &r, &g, &b);  
6         // Application de la formule de luminance  
7         Uint8 gray = 0.299*r + 0.587*g + 0.114*b;  
8         pixels[i] = SDL_MapRGB(surface->format, gray, gray, gray);  
9     }  
10 }
```

2.3 Réduction du Bruit (Denoising)

Une image brute contient souvent du bruit : pixels parasites, poussière sur le scanner, grain de la photo. Ces défauts peuvent être interprétés à tort comme des points sur les "i" ou des parties de lettres par l'OCR. Pour contrer cela, nous appliquons un filtre médian. Contrairement au filtre moyenneur qui floute l'image, le filtre médian préserve les contours nets tout en supprimant les valeurs aberrantes (le bruit poivre et sel).

Pour chaque pixel, nous considérons son voisinage (par exemple, une fenêtre 3×3). Nous trions les valeurs de gris des pixels de cette fenêtre et remplaçons la valeur du pixel central par la valeur médiane. Cette méthode est particulièrement efficace pour éliminer les bruits impulsifs sans altérer la lisibilité des caractères.

2.4 Correction Automatique de la Rotation (Deskew)

Les images prises avec un téléphone ne sont jamais parfaitement droites. Une rotation de quelques degrés suffit à casser la structure de ligne/colonne de la grille. Il est donc critique de redresser l'image automatiquement.

2.4.1 Approche par Projection de Variance

Nous avons implémenté une méthode basée sur l'histogramme de projection. Le principe est le suivant :

- Si le texte est horizontal, la projection des pixels noirs sur l'axe vertical crée des pics très marqués (les lignes de texte) et des creux (les interlignes). La variance de cet histogramme est donc **maximale**.
- Si le texte est penché, les lignes se mélangent lors de la projection. L'histogramme est plus plat, la variance diminue.

L'algorithme teste donc une plage d'angles (de -45° à $+45^\circ$) et cherche l'angle qui maximise cette variance.

2.4.2 Rotation Matricielle

Une fois l'angle θ trouvé, nous appliquons une rotation. Pour éviter les trous dans l'image finale, nous utilisons la technique du "Backward Mapping". Pour chaque pixel de destination (x', y') , nous calculons son antécédent dans l'image source (x, y) :

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x' - x_c \\ y' - y_c \end{pmatrix} + \begin{pmatrix} x_c \\ y_c \end{pmatrix} \quad (2.2)$$

Où (x_c, y_c) est le centre de rotation.

2.5 Binarisation Adaptative

La binarisation consiste à transformer l'image en noir et blanc pur. Un seuillage simple (si pixel > 127 alors blanc) ne fonctionne pas à cause des ombres et de la luminosité variable.

2.5.1 Méthode de Niblack

Nous utilisons la méthode de Niblack qui calcule un seuil local pour chaque pixel. Pour un pixel (x, y) , le seuil T est calculé en fonction de la moyenne μ et de l'écart-type σ de son voisinage :

$$T(x, y) = \mu(x, y) + k \times \sigma(x, y) \quad (2.3)$$

Avec $k = -0.2$ généralement.

2.5.2 Optimisation : Image Intégrale

Calculer la moyenne locale pour chaque pixel est très lent ($O(N \times r^2)$). Pour optimiser cela, nous utilisons une **Image Intégrale**. C'est une structure de données qui permet de calculer la somme d'une zone rectangulaire en seulement 4 opérations, quelle que soit la taille de la zone.

$$Sum(A, B, C, D) = I(D) + I(A) - I(B) - I(C) \quad (2.4)$$

Grâce à cette optimisation, le traitement est quasi-instantané même sur des images haute résolution.

Chapitre 3

Détection et Segmentation

Par Mezroui Ayoub

3.1 Analyse des Composantes Connexes

Une fois l'image binarisée, elle ne contient que des 0 et des 1. L'ordinateur ne "voit" pas encore de lettres. La première étape de la segmentation est l'extraction des composantes connexes (Connected Component Labeling - CCL). L'algorithme parcourt l'image et regroupe les pixels noirs adjacents pour former des "blobs". Chaque blob correspond potentiellement à une lettre.

3.1.1 Structure de Données

Nous stockons chaque composante détectée dans une structure C :

```
1 typedef struct {  
2     int x, y;           // Coin sup rieur gauche  
3     int w, h;           // Largeur et Hauteur  
4     int surface;        // Nombre de pixels  
5     int label;          // Identifiant unique  
6 } BoundingBox;
```


3.2 Filtrage et Nettoyage

Beaucoup de composantes détectées ne sont pas des lettres (bruit, taches, bordures). Il faut les filtrer. Nous appliquons plusieurs critères :

- **Taille minimale** : Si surface < 20 pixels, c'est du bruit $\rightarrow$ Suppression.
- **Taille maximale** : Si la boîte couvre 80% de l'image, c'est une bordure $\rightarrow$ Suppression.
- **Ratio** : Une lettre est généralement inscrite dans un carré. Si $Largeur / Hauteur > 4$, c'est probablement une ligne $\rightarrow$ Suppression.

3.3 Séparation Grille / Liste de Mots

C'est le défi principal de cette partie. L'image contient deux zones de texte distinctes : la grille (dense, structurée) et la liste de mots (alignée, espacée). Comment les différencier automatiquement ?

3.3.1 Algorithme de Clustering

Nous utilisons une approche de regroupement spatial (Clustering). Si deux lettres sont proches (distance inférieure à un seuil D), elles appartiennent au même groupe. Nous utilisons l'algorithme **Union-Find** pour gérer ces groupes efficacement.

```
1 // Pseudo-code Union Find pour le clustering
2 void unite(int a, int b) {
3     a = find(a);
4     b = find(b);
5     if (a != b) {
6         parent[b] = a;
7         size[a] += size[b];
8     }
9 }
```

Une fois les groupes formés :

- Le groupe le plus grand et le plus dense est identifié comme étant la **Grille**.
- Les autres groupes significatifs sont identifiés comme la **Liste de Mots**.

3.4 Découpage des Caractères (Slicing)

Une fois que nous avons identifié les blocs correspondant à la grille, nous devons extraire chaque caractère individuellement. Parfois, deux lettres peuvent se toucher à cause d'une binarisation imparfaite ou d'une police d'écriture serrée. Pour résoudre ce problème, nous analysons l'histogramme de projection verticale à l'intérieur de chaque ligne de texte. Les minima locaux de cet histogramme indiquent les espaces probables entre les caractères, permettant de couper les lettres fusionnées.

3.5 Reconstruction de la Grille

Une fois la zone de la grille isolée, il faut reconstruire la matrice logique $M[Ligne][Colonne]$. Nous analysons les coordonnées des lettres pour déduire le pas moyen (écart moyen entre deux lettres).

$$Col = round((x - x_{min})/Step_X) \quad (3.1)$$

$$Row = round((y - y_{min})/Step_Y) \quad (3.2)$$

Enfin, chaque case est découpée en une image de 50×50 pixels, centrée et redimensionnée, prête à être envoyée au réseau de neurones.

Chapitre 4

Intelligence Artificielle et OCR

Par Chahine Nabi

4.1 Introduction

La reconnaissance optique de caractères (OCR) est assurée par un réseau de neurones artificiels. Conformément aux contraintes du projet, ce réseau a été implémenté entièrement en C, sans l'aide de bibliothèques de Machine Learning comme TensorFlow ou PyTorch.

4.2 Architecture du Modèle

Nous avons opté pour un Perceptron Multicouche (MLP - Multi-Layer Perceptron), une architecture classique mais efficace pour la classification d'images simples comme des caractères.

4.2.1 Topologie

Le réseau est constitué de 3 couches de neurones :

- **Couche d'Entrée (Input Layer)** : 2500 neurones. Cela correspond à nos images de taille 50×50 pixels aplaties en un vecteur 1D. Chaque neurone reçoit la valeur d'un pixel (0 pour noir, 1 pour blanc).
- **Couche Cachée (Hidden Layer)** : 2 couches de 64 neurones. Ces couches permettent au réseau d'apprendre des caractéristiques non linéaires (courbes, boucles des lettres). La fonction d'activation utilisée est la Sigmoid.
- **Couche de Sortie (Output Layer)** : 52 neurones. Chaque neurone correspond à une classe de caractère (A-Z majuscule et a-z minuscule). Le neurone avec la valeur la plus élevée indique la lettre reconnue.

[Image of Neural Network Architecture Diagram]

4.3 Mathématiques du Réseau

4.3.1 Propagation Avant (Forward Propagation)

Pour une entrée X , la sortie de chaque couche l est calculée comme suit :

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]} \quad (4.1)$$

$$A^{[l]} = \sigma(Z^{[l]}) \quad (4.2)$$

Où W est la matrice des poids, b le vecteur de biais, et σ la fonction d'activation sigmoïde :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

4.3.2 Rétropropagation (Backpropagation)

L'apprentissage consiste à minimiser l'erreur entre la prédiction du réseau et la vraie lettre. Nous utilisons l'algorithme de descente de gradient. Le gradient de la fonction de coût C par rapport aux poids est calculé en remontant le réseau de la fin vers le début (Règle de la chaîne).

$$\frac{\partial C}{\partial w_{jk}} = \frac{\partial C}{\partial a_k} \cdot \sigma'(z_k) \cdot a_j \quad (4.3)$$

Ces formules ont été implémentées manuellement en C via des multiplications de matrices.

4.4 Dataset et Entraînement

Pour entraîner notre réseau, nous avons constitué un jeu de données (dataset) contenant plusieurs milliers d'images de lettres dans différentes polices (Arial, Times New Roman, Comic Sans, etc.) et avec différents niveaux de bruit. L'entraînement a duré environ 50 époques (passages complets du dataset) pour atteindre un taux de précision satisfaisant ($> 95\%$). Les poids synaptiques finaux sont sauvegardés dans un fichier '.w' chargé au lancement de l'application.

4.5 Preuve de Concept : Le XOR

Avant de nous attaquer à la reconnaissance de caractères, nous avons validé notre implémentation du réseau de neurones en résolvant le problème du XOR (OU exclusif). Le XOR est un problème de classification non linéaire simple qui ne peut pas être résolu par un simple perceptron. Notre réseau, configuré avec 2 entrées, une couche cachée de 2 neurones et une sortie, a réussi à apprendre la table de vérité du XOR après quelques milliers d'itérations, confirmant ainsi le bon fonctionnement de notre algorithme de rétropropagation.

Chapitre 5

Résolution de la Grille (Solver)

Par Chahine Nabi

Une fois que l'OCR a transformé l'image en une matrice de caractères et une liste de mots (strings), le problème devient purement algorithmique.

5.1 Algorithme de Recherche

Pour chaque mot de la liste, nous devons le trouver dans la grille. L'algorithme parcourt chaque case de la grille. Si la lettre correspond à la première lettre du mot cherché, une recherche directionnelle est lancée.

5.1.1 Les 8 Directions

Un mot peut être écrit horizontalement, verticalement ou en diagonale, et dans les deux sens (endroit/envers). Cela fait 8 directions possibles, représentées par des vecteurs de direction (dx, dy) :

- $(1, 0)$: Droite
- $(1, 1)$: Diagonale Bas-Droite
- $(0, 1)$: Bas
- $(-1, 1)$: Diagonale Bas-Gauche
- etc.

```
1 int solve(char grid[N][N], char* word) {
2     for(int y=0; y<N; y++) {
3         for(int x=0; x<N; x++) {
4             if(grid[y][x] == word[0]) {
5                 // Check all 8 directions
6                 for(int d=0; d<8; d++) {
7                     if(check_word(grid, word, x, y, dx[d], dy[d]))
8                         return FOUND;
9                 }
10            }
11        }
12    }
13    return NOT_FOUND;
14 }
```

5.2 Gestion des Erreurs OCR

L'OCR n'est pas parfait. Il peut confondre un 'O' et un 'Q' ou un 'I' et un '1'. Pour rendre le solver robuste, nous avons envisagé d'utiliser la distance de Levenshtein, permettant de trouver un mot même si une lettre est fausse, mais cette fonctionnalité a été jugée trop coûteuse en temps de calcul pour la version finale. L'approche retenue est la correction manuelle via l'interface utilisateur.

Chapitre 6

Interface Utilisateur et Intégration

Par Yanis Jerjini

6.1 Conception de l'Interface

L'interface utilisateur (GUI) est l'élément qui unifie le projet. Elle a été réalisée par Yanis en utilisant la bibliothèque **GTK3** (GIMP Toolkit). L'objectif était de créer une interface moderne, intuitive et réactive.

6.1.1 Architecture Logicielle

Nous avons séparé la logique du programme de la vue.

- **Glade (XML)** : La structure de la fenêtre (boutons, zones d'images, menus) est décrite dans un fichier XML. Cela permet de modifier le design sans toucher au code C.
- **CSS Provider** : Pour éviter l'aspect "gris" par défaut de GTK, Yanis a injecté du code CSS pour styliser les éléments (bordures arrondies, couleurs, effets de survol).

6.2 Fonctionnalités Implémentées

L'interface propose un workflow linéaire pour l'utilisateur :

6.2.1 1. Chargement et Affichage

L'utilisateur charge une image via un `GtkFileChooser`. L'image est redimensionnée pour s'adapter à la fenêtre tout en conservant son ratio d'aspect, puis affichée dans un widget `GtkImage`.

6.2.2 2. Visualisation des Étapes Intermédiaires

Pour le débogage et la démonstration lors de la soutenance, l'interface permet de voir les étapes du traitement :

- Bouton "Voir Niveaux de Gris"
- Bouton "Voir Binarisation"
- Bouton "Voir Rotation"

Ces fonctionnalités appellent directement les fonctions du module d'Issac et rafraîchissent l'affichage.

6.2.3 3. Édition et Correction

Une fonctionnalité clé développée par Yanis est la possibilité de corriger l'OCR. Si le réseau de neurones se trompe, l'utilisateur peut cliquer sur une case de la grille affichée à l'écran et taper la bonne lettre au clavier. Cela met à jour la matrice interne avant de lancer le Solver.

6.3 Dessin Vectoriel avec Cairo

Pour afficher les résultats (les mots entourés en couleur sur l'image), nous ne modifions pas les pixels de l'image. Nous utilisons la librairie **Cairo** pour dessiner des formes vectorielles "par-dessus" l'image. Cela permet d'avoir des traits lisses, transparents et colorés indiquant où les mots ont été trouvés.

Chapitre 7

Conclusion

Ce projet de semestre 3 a été un défi technique majeur. Partir d'une image brute pour arriver à une compréhension sémantique (les mots) demande de maîtriser une chaîne complexe de traitements.

7.1 Bilan Technique

- **Prétraitement** : Le module d'Issac est robuste aux rotations jusqu'à 45 degrés et aux mauvais éclairages grâce à la binarisation adaptative.
- **Segmentation** : L'algorithme d'Ayoub sépare correctement la grille et la liste dans 90% des cas testés.
- **IA** : Le réseau de neurones de Chahine, bien que minimaliste, offre des performances suffisantes pour des polices standards.
- **Intégration** : L'interface de Yanis rend le tout utilisable et fluide.

7.2 Améliorations Possibles

Si nous avions plus de temps, nous pourrions :

1. Utiliser un réseau de neurones convolutif (CNN) pour améliorer la reconnaissance des lettres manuscrites.
2. Gérer les grilles déformées (pages courbées) via des transformations géométriques non linéaires.
3. Porter l'application sur mobile (Android/iOS).

Ce projet nous a permis de consolider nos connaissances en C et de découvrir la réalité du développement logiciel en équipe avec Git.

Annexe A

Guide d'Utilisation

A.1 Compilation

Pour compiler le projet, placez-vous à la racine et exécutez :

```
1 $ make
```

Cela générera l'exécutable `ocr_solver`.

A.2 Exécution

```
1 $ ./ocr_solver
```

Une fois l'interface lancée : 1. Cliquez sur "Ouvrir" (icône dossier). 2. Sélectionnez une image de test dans le dossier **assets/**. 3. Cliquez sur "Pré-traitement" pour voir l'image nettoyée. 4. Cliquez sur "Résoudre" pour lancer l'IA et l'algorithme.

Annexe B

Bibliographie

1. Cours de Traitement d'Image EPITA S3.
2. "Digital Image Processing", Gonzalez & Woods.
3. Documentation officielle GTK+ 3.
4. "Neural Networks and Deep Learning", Michael Nielsen.
5. Cahier des charges du projet OCR, David Bouchet.